Laborator 4

Sumar

Map
Generics
toString()
equals()
hashCode()

4.1 Map

java.util.Map face legatura dintre o cheie si o valoare. In acest tip de structura stocam perechi cheie-valoare. Interfata java.util.Map are urmatoarele implementari:

java.util.HashMap
java.util.Hashtable
java.util.EnumMap
java.util.IdentityHashMap
java.util.LinkedHashMap
java.util.Properties
java.util.TreeMap
java.util.WeakHashMap

Operatii:

```
Map mapA = new HashMap();
mapA.put("key1", "element 1");
mapA.put("key2", Integer.valueOf(123));
mapA.put(null, "elem");           //HashMap permite cheie nula.
mapA.put("key3", null);           //valori nule sunt permise
map.replace("key1", "new value"); //daca se gaseste cheia,
                                  //atunci se suprascrie valoarea
mapA.put("key3", "newVal"); //dupa executia acestei metode,
                            //cheii key3 i se asociaza obiectul "newVal"
Object value1 = mapA.get("key1"); //citirea valorii  asociate cheii "key1"
Object value2 = mapA.get("key2");
mapA.remove("key1"); //eliminare element cheie-valoare
mapA.clear(); //eliminarea tuturor elementelor
Object value3 = map.getOrDefault("key4", "default value"); //daca nu
                      //gaseste cheia "key4", returneaza "default value"
boolean hasKey = mapA.containsKey("key1");//verifica existenta unei chei
boolean hasValue = mapA.containsValue("element 1");//verifica existenta
                                                   //unei valori
int entryCount = mapA.size();

Map mapB = new TreeMap();
mapB.putAll(mapA);   //inserarea tuturor elementelor altui obiect Map

Iterator iterator = mapA.keySet().iterator();
while(iterator.hasNext(){  //parcurgere Map folosind iterator
  Object key   = iterator.next();
  Object value = mapA.get(key);
}
```

Laborator 4

```
for(Object key : mapA.keySet()) { //parcurgere Map folosind for-each
    Object value = mapA.get(key);
}

Stream<String> stream = map.keySet().stream(); //parcurgere cu stream si
                                               //lambda functions(java 1.8+)
stream.forEach((value) -> {
    System.out.println(value);
});

Iterator iterator = map.values().iterator(); //parcurgere a valorilor
while(iterator.hasNext()) {
    Object nextValue  iterator.next();
}

for(Object value : mapA.values()){ //parcurgere valori cu for-each
    System.out.println(value);
}

Stream<String> stream = mapA.values().stream(); //parcurgere valori
                                        //cu stream si lambda functions
stream.forEach((value) -> {
    System.out.println(value);
});

Set entries = map.entrySet(); //parcurgerea intrarilor in Map, cu iterator
Iterator iterator = entries.iterator();
while(iterator.hasNext()) {
    Map.Entry entry = (Map.Entry) iterator.next();
    Object key = entry.getKey();
    Object value = entry.getValue();
}

for(Object entryObj : map.entrySet()){
    Map.Entry entry = (Map.Entry) entryObj;
    Object key = entry.getKey();
    Object value = entry.getValue();
}
```

4.2 Generics

4.2.1 Necesitate

Observati necesitatea conversiei de tip (**cast**) pentru a putea utiliza variabila x:

```
List myIntList = new LinkedList();
myIntList.add(new Integer(0));
Integer x = (Integer) myIntList.iterator().next();
```

Pentru codul de mai sus se remarca urmatoarele dezavantaje :

- este ingreunată citirea codului
- apare posibilitatea unor erori la executie, in momentul in care in listă se introduce un obiect care nu este de tipul Integer.

Genericitatea elimina aceste probleme, si codul cu generice va arata asa:

```
List<Integer> myIntList = new LinkedList<Integer>();
myIntList.add(new Integer(0));
Integer x = myIntList.iterator().next();
```

4.2.2 Type-Safe la compilare

In limbajul java, array-urile sunt type-safe. Urmatorul cod comporta eroare la compilare:

```
public class NG {
    public static void main (String[] args) {
        String name[] =new String[500];
        name[0] = "Ioan Popa";
        name[1] = "Vasile Alecsandri";
        name[2] = Integer.valueOf(100);
    }
}
```

```
Task :compileJava FAILED
[Incubating] Problems report is available at: file:///C:/.../problems-report.html
1 actionable task: 1 executed
C:\...\src\main\java\NG.java:6: error: incompatible types: Integer cannot be converted
to String
        name[2] = Integer.valueOf(100);
                                  ^
```

Listele in schimb nu sunt type-safe. Urmatorul cod se compileaza, iar o exceptie este aruncata la runtime:

```
public class WG {
    public static void main (String[] args) {
        ArrayList al =new ArrayList();
        al.add("Ioan Popa");
```

```
        al.add("Vasile Alecsandri");
        al.add(Integer.valueOf(10));

        String name1 = (String)al.get(0);
        String name2 = (String)al.get(1);
        String name3 = (String)al.get(2);
    }
}
```

Compilare cu success.
Rezultatul rularii:

```
Task :WG.main() FAILED

[Incubating] Problems report is available at: file:///C:/.../problems-report.html
2 actionable tasks: 2 executed
Exception in thread "main" java.lang.ClassCastException: class java.lang.Integer cannot
be cast to class java.lang.String (java.lang.Integer and java.lang.String are in module
java.base of loader 'bootstrap')
    at WG.main(WG.java:12)
```

4.2.3 Definire Generice

Extras din definitia interfetelor **List** si **Iterator**:

```
public interface List<E> {
    void add(E x);
    Iterator<E> iterator();
}

public interface Iterator<E> {
    E next();
    boolean hasNext();
    void remove();
}
```

Sintaxa <E> (poate fi folosita orice litera) este folosita pentru a defini tipuri formale in cadrul interfetelor. Aceste tipuri pot fi folosite in mod asemanator cu tipurile uzuale, cu anumite restrictii totusi. In momentul in care invocam o structura generica ele vor fi inlocuite cu tipurile efective utilizate in invocare.

Pentru urmatorul apel

```
ArrayList<Integer> myList = new ArrayList<Integer>();
Iterator<Integer> it = myList.iterator();
```

tipul formal E a fost înlocuit (la compilare) cu tipul efectiv Integer.

4.2.4 Wildcard

```
void printCollection(Collection<?> c) {
    for (Object e : c) {
        System.out.println(e);
    }
}
```

Codul de mai sus permite ca functia printCollection() sa poata fi apelata cu parametru o colectie cu orice tip de date.
Intr-o colectie definita cu wildcard nu vom putea adauga elemente aribitrare:

```
Collection<?> c = new ArrayList<String>();  // Operatie permisa
c.add(new Object());                        // Eroare la compilare
```

4.2.5 Bounded wildcards

Folosim acest concept cand dorim sa adaugam anumite restrictii pe wildcard:

```
class Animal {
    protected String name = "Animal";

    public String getName() {
        return name;
    }
}

class Cat extends Animal {
    public Cat() {
        name = "Cat Animal";
    }
}

class Mouse extends Animal {
    public Mouse() {
        name = "Mouse Animal";
    }
}

class MyApplication {
                                    // Folosire "bounded wildcards"
    public static void listAnimal(List<? extends Animal> theList) {
        for(Animal item : theList)
            System.out.println(item.getName());
    }

    public static void main(String[] args) {
        List<Animal> aList = new ArrayList<Animal>();

        aList.add(new Cat());
        aList.add(new Mouse());
        aList.add(new Animal());

        listAnimal(aList);   // Se va afisa: "Cat Animal", "Mouse Animal", "Animal"
    }
}
```

4.2.6 Metode generice

```
static <T> void copy(T[] a, Collection<T> c) {
    for (T o : a) {
        c.add(o);
    }
}
```

```
// copiere elemente dintr-o lista în alta lista
static <T> void copy(List<T> dest, List<? extends T> src) { ... }
```

4.3 toString

Toate clasele mostenesc metoda toString de la clasa Object, iar implementarea acesteia in clasa Object este:

```
public String toString() {
    return getClass().getName() + "@" + Integer.toHexString(hashCode());
}
```

IntelliJ Idea poate scrie aceasta functie atunci cand dorim (ALT+INSERT > toString, sau click-dreapta > Generate > toString, sau top menu Code > Generate > toString).

Aceasta metoda este utilizata pentru convertirea oricarui obiect din memorie la o reprezentare String. Ne dorim sa implementam aceasta metoda daca vrem sa afisam obiectele noastre, sau dorim sa salvam obiectele ca text intr-un mediu de stocare.

4.4 equals, hashCode

Si metodele equals() si hashCode() sunt definite in clasa Object si deci toate instantele vor mosteni aceste doua metode.

Ne dorim sa suprascriem equals si hashCode atunci cand dorim ca doua instante diferite din memorie sa fie egale. Pentru doua obiecte egale din memorie functia hashCode trebuie sa returneze exact aceeasi valoare.

Nu este obligatoriu ca pentru doua obiecte neegale (equals() returneaza false) ca functia hashCode() sa returneze valori distincte. Adica doua obiecte neegale pot returna aceeasi valoare hashCode(). Totusi se recomanda ca pentru obiecte neegale hashCode() sa returneze valori neegale, asta pentru a imbunatati performanta HashTable-urilor cand aceste obiecte sunt inserate in acest tip de structuri.

4.5 Probleme

4.5.1 Creati subpachetul lab4 in proiectul **ProiectareSoftware,** si adaugati clasele necesare pentru cele de mai jos.

Creati clasa **AppLab4**. Scrieti ***main*** si apoi apasati tab – prin asta Intellij Idea ar trebui sa creeze metoda main.

Laborator 4

Declarati urmatoarele doua instante map:

```
    HashMap<String, Integer> varste = new HashMap<>();
    varste.put("Ioan", 21);
    varste.put("Maria", 22);
    varste.put("Victor", 20);
    varste.put("Simina", 20);
    varste.put("Marius", 21);
    varste.put("Mihai", 21);
    varste.put("Daniela", 23);

    Map<String, String> adrese = Map.of("Ioan", "Sibiu", "Maria", "Bucuresti", "Victor",
"Cluj","Simina", "Alba-Iulia","Marius", "Medias", "Mihai", "Cisnadie","Daniela", "Sibiu");
```

a) Afisati continutul instantei **varste**. Adaugati Vlad – 19 ani si Iulia-19 ani. Afisati
b) Declarati clasa Tanar, care sa aiba urmatoarele campuri:

```
private String name;
private int age;
private String address;
```

Scrieti constructor care sa aiba parametri pentru toate cele trei campuri. Scrieti metoda toString() (Tip: Alt+Insert > Constructor)

c) Parcurgeti cele doua instante Map de mai sus, si generati o a treia de tipul:

```
HashMap<String, Tanar> tineri;
```

Introduceti in aceasta instanta, tinerii din instanta **varste.** Folositi prenumele pentru cheie. Adaugati si adresa la momentul instantierii fiecarui obiect de tipul **Tanar**.

d) Afisati in consola continutul instantei **tineri.**

Comiteti codul, si push-uiti-l pe GitHub.

4.5.2. Pentru proiectul Studenti, aveti un nou fisier **note_anon.txt** din care se citesc notele pentru studenti. Studentii sunt scrisi sub forma de numar matricol. Cititi continutul acestui fisier si alocati notele catre fiecare Student in O(1).

Pentru aceasta

- adaugati campul nota in clasa Student, adaugati metoda setter
- implementati equals, hashCode. Extindeti toString() astfel incat sa includa si nota.
- creati un HashMap din setul de Studenti cititi din fisierul de data trecuta.

- cititi noul fisier note_anon.txt, si pentru fiecare numar matricol citit actualizati instanta Sudent precizand nota (apelul metodei setter)
- parcurgeti Map-ul cu student si afisati fiecare element

Asigurati-va ca fisierul din care cititi studentii numarul matricol este unic per student. Ex:

1027,Paul,Mohanu,TI132/1
1028,Mihai,Ionescu,TI132/1
1025,Andrei,Popa,ISM141/2
1024,Ioan,Mihalcea,ISM141/1
1026,Anamaria,Prodan,TI131/1
1029,Bianca,Popescu,TI131/1

Iar fisierul note_anon.txt

1024,9.80
1025,8.70
1026,8.90
1027,5.40
1028,6.20
1029,9.10

Comiteti codul, si push-uiti-l pe GitHub.

4.5.3 Tema aplicatia “Studenti”

Scrieti o noua metoda statica in clasa principala care sa returneze nota unui Student in O(1). Se trimit ca parametri prenumele, numele studentului, si ultima instanta Map.

Apelati aceasta metoda si afisati rezultatul. Apelati pentru doi studenti:

```
float notaM = gasesteNota("Bianca", "Popescu", tineri);
float notaN = gasesteNota("Ioan", "Popa", tineri);
```

Daca studentul nu este gasit, returneaza 0.0.

Comiteti codul, si push-uiti-l pe GitHub cu comentariul “Tema Laborator 4”

Tip: in cadrul metodei, construiti un HashMap<String, Student> cu datele din Map-ul tineri.

Cheia ar putea fi: “prenume-nume” (sau orice altceva care sa include numele si prenumele)

4.6 Referinte

HashMap
https://www.programiz.com/java-programming/hashmap

Generics in Java
https://www.geeksforgeeks.org/java/generics-in-java/

Type Erasure in Java
https://www.geeksforgeeks.org/java/type-erasure-java/

Generics with Collections
https://www.geeksforgeeks.org/advance-java/generics-with-collections/

Non-generic Vs Generic Collection In Java
https://www.geeksforgeeks.org/java/non-generic-vs-generic-collection-in-java/